Mediator, Not Authority: The Architectural Role of LLMs in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 24 April 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to extract, from the source paper's distributed treatment of AI-as-substrate-mediator, a single independently citable definition of the architectural role that LLMs play in CKS systems, and to articulate the operational and cross-claim consequences of that role.

Abstract

The CKS pattern names AI-as-substrate-mediator as one of its six architectural commitments and identifies it (§4.2) as the cross-claim spine connecting the other five. The commitment is load-bearing but distributed across §4.1, §4.2, and §11.3, with no single landing site. This note formalizes the commitment as a single architectural role with a five-property definition; distinguishes it from four adjacent roles LLMs play in other architectures (terminal producer, autonomous agent, source of truth, substrate client); states the operational requirements the role imposes; traces how the role enables each of the other five commitments; and provides an operational test for whether a system instantiates the AI-as-substrate-mediator commitment as the source paper uses the term.

1. Why the mediator role needs to be named

AI-as-substrate-mediator is one of six architectural commitments, but it is the only one whose operational content is implicit in the relationship between two other commitments — the substrate/LLM governance boundary (§4.1) and the substrate as source of truth (§11.3) — rather than stated as a single role. The source paper acknowledges this in §4.2, where the commitment is identified as the *cross-claim spine* connecting Claim 2 to Claim 6 across single-human and multi-human contexts. Removing the commitment does not weaken any single claim in particular; it removes the role that ties the other five into one design posture rather than a list of independent properties. Without a single landing site, downstream work has no independent citation target for the LLM's role.

The other reason to name the role is that the boundary of what LLMs are authorized to do otherwise remains implicit. Implementations that grant LLMs unauthorized powers — silent overwrite of substrate content, in-LLM memory of substrate state treated as authoritative, modification of orchestration rules without human action — can describe themselves as CKS-aligned because no single statement directly forbids them. A definition with five properties and an operational test makes those out-of-bounds powers explicitly identifiable.

2. The definition

In the CKS pattern, an LLM operates as a **substrate mediator** if and only if all of the following five properties hold for every operation the LLM performs in the system, at all times during the system's existence:

1. **Substrate-content reads as primary state.** The LLM reads from substrate content as its primary source of state for the operation. In-context information not drawn from the substrate is admissible only insofar as it does not serve as a substitute for substrate state the operation depends on.

2. **Substrate-content writes under orchestration rules.** The LLM writes to substrate content only under orchestration rules authored by humans. The form of the rules is not specified by the architecture; what is specified is that they are human-authored and the LLM operates within them rather than authoring or modifying them.

3. **No substrate-relevant state outside the substrate.** The LLM does not hold substrate-relevant state outside the substrate. This rules out three forms of out-of-substrate state treated as authoritative: agent memory persisted across LLM invocations, per-session storage that carries substrate-relevant state forward without being written into the substrate, and in-weight memory of substrate content treated as the answer to “what is the case.” Substrate-relevant state lives in the substrate.

4. **No authority over substrate content.** The LLM does not exercise authority over substrate content. It cannot silently overwrite substrate content, collapse contradictions the substrate preserves, or modify the orchestration rules under which it operates. Authority sits with humans, per the human-governed commitment; the LLM executes within the boundaries that authority defines.

5. **LLM writes recorded as substrate content with attribution.** When an LLM operation results in a write to substrate state, the write is itself recorded in the substrate as substrate content, with attribution sufficient to identify it as LLM-authored and to associate it with the orchestration rule under which it was produced.

Properties (1) and (2) follow from §4.1's governance boundary; (3) follows from §11.3; (4) follows from the human-governed commitment; (5) follows from §3.1's path-retraceability commitment.

3. The governance boundary the role defines

The mediator role is the operational expression of the substrate/LLM division of labor §4.1 commits to: the substrate handles what is deterministic, human-governed, and auditable; the LLM handles what is high-dimensional and non-deterministic — reasoning over unstructured inputs, language understanding, drafting, interpretation. The split is drawn at the governance boundary, not the capability boundary; each side does the work it should be authorized to do given the architecture's commitments to authority, auditability, and conflict preservation.

What the mediator role adds is the *operational hinge* that closes the split. The LLM is authorized to do the high-dimensional reasoning the substrate cannot encode, and bound to express the results as substrate writes, where they become deterministic substrate content subject to inspection, modification, and override. Without the mediator role, §4.1 is an allocation of work; with it, the split becomes a closed loop — high-dimensional work happens, its outputs land in the substrate, and the substrate carries them forward as the artifact subsequent operations read from.

4. What the mediator role is not

Each of the following is a real role LLMs play in some other architecture, and each is the role an LLM plays in CKS *unless* the mediator role is explicitly committed to.

Not terminal producer. In the conventional pattern, an LLM receives a request, produces an artifact, and the artifact is the deliverable; the substrate, if any, is incidental scaffolding. In CKS, the substrate is the primary artifact; LLM outputs are valuable insofar as they update or operate over it, and what is delivered across sessions is the substrate's persistent content rather than any single LLM output produced en route.

Not autonomous agent. In the agent pattern, an LLM holds goals, plans, and intermediate state across multiple steps and exercises judgment about what actions to take. The state lives in agent memory — outside the substrate, treated as authoritative for the duration of the agent's operation. In CKS, the LLM does not hold substrate-relevant state across steps; substrate state is in the substrate, and the LLM operates over it. Multi-step reasoning *within* a single cell invocation is admissible, but does not produce authority over substrate content beyond what orchestration rules authorize, and its intermediate state is not substrate state.

Not source of truth. In some patterns, an LLM's parametric knowledge or in-context reasoning is treated as the authoritative answer to “what is the case.” In CKS, “what is the case” is recovered from the substrate. The §11.3 commitment is the operative one: substrate is source of truth, and LLM operations are recorded back into it as substrate content rather than treated as having produced the truth themselves.

Not substrate client. In the epistemically-structured-substrate architectures the source paper distinguishes CKS from at §5.2 — most directly OIDA — the LLM consumes structured memory to answer queries but does not write into the substrate, and is not bound by orchestration rules over substrate writes. The architecture is read-for-consumption rather than read-and-write-under-rules.

CKS's mediator role is bidirectional: the LLM reads, operates, and writes results back as substrate content with attribution. A read-only consumer satisfies properties (1), (3), and (4) trivially but not (2) or (5), and is not a CKS substrate mediator regardless of how rich the structured memory it consumes.

5. Operational requirements

For the mediator role to be meaningfully implemented, the system must satisfy four operational requirements — the conditions under which the §2 definition is exercisable.

Substrate-content readability. The LLM must be able to read substrate content at cell execution time. This is the third of the three minimal requirements §7.1 names for tool-agnosticism. Without it, property (1) cannot hold.

Orchestration-rule expressibility. The orchestration rules under which the LLM writes must be expressible in a form the LLM can operate under. The source paper does not commit to a specific form; it may be natural-language prompts, structured configuration, executable specifications, or any combination, provided the rules are human-authored.

Write attribution. Substrate content the LLM writes must be attributable as such. The substrate must record that a given piece of content was written by an LLM, under which orchestration rule, in which cell execution. Without this, property (5) lands in storage but not in practice. The path-retraceability commitment §3.1 imports is the substrate-level mechanism this requirement relies on.

Boundary inspectability. The boundary between LLM-authored and human-authored substrate content must be inspectable by a human exercising the inspect right. It is not sufficient for the substrate to *carry* attribution metadata; the metadata must be exposable to the human reading the substrate.

6. The cross-claim spine

§4.2 identifies AI-as-substrate-mediator as the cross-claim spine connecting the other commitments. None of the relationships below adds a new commitment; each names the work the mediator role does in making one of the other five commitments operationally coherent.

Enabling human-governed. The human-governed commitment requires that humans retain authority over substrate content and orchestration rules at all times. That is satisfiable only if the LLM does not exercise the authority humans hold. Property (4) makes the non-exercise architectural rather than procedural: the LLM is structurally not the holder of authority. A system that grants the LLM authority and asks it to defer is governed by promise; a system that does not grant the LLM authority is governed by architecture.

Enabling conflict preservation. Conflict preservation requires that contradictions in the substrate are not silently collapsed by any operation over it. The collapse-prohibition is enforceable only if the LLM does not have the authority to perform it, and property (4) does that work. Where collapse is appropriate, it must be authorized by a human directly modifying the substrate or by an orchestration rule humans have authored. Either path keeps collapse under human authority; neither lets the LLM perform it on its own initiative.

Enabling tool-agnosticism. Tool-agnosticism requires that the substrate run in any environment satisfying the three minimal requirements §7.1 names: persistent structured state, human read/write access, and LLM access to substrate content. The third is what the mediator role exercises. Because the LLM operates over the substrate through standard read and write operations rather than specialized runtime middleware, the substrate can live in any environment supporting those operations. Holding substrate-relevant state in agent memory or a specialized runtime would re-couple the substrate to that runtime; property (3) keeps the coupling at the substrate layer.

Enabling linear-cost scaling. The linear-cost commitment (§6.1) inherits the database-like cost curve from the substrate infrastructure. The inheritance is sound only if LLM work scales with cell executions rather than substrate size. Property (3) makes that the case: the LLM does not maintain global state proportional to substrate size, so its work per cell execution does not grow with how much the substrate carries. An in-weight or agent-memory model of substrate contents would couple LLM cost to substrate size; the mediator role decouples them.

Enabling the human-AI-human extension. The Claim 6 extension to multi-human coordination does not introduce a new architectural role for the LLM; it carries the mediator role into a context where multiple humans in different roles read, write, and audit the same substrate. The same five properties constrain the LLM in single-human and multi-human use, with no role-specific exception. The role-symmetry of the mediator commitment — no participant role is privileged, because the LLM operates over the substrate rather than for any participant — is what lets the extension stand on the core theory rather than as a separately defended architecture.

7. Operational test

A system implements the AI-as-substrate-mediator commitment if and only if all of the following hold for every LLM operation in the system, at all times:

1. The LLM reads from substrate content as its primary source of state.
2. The LLM writes to substrate content only under orchestration rules authored by humans.
3. The LLM does not hold substrate-relevant state outside the substrate (no agent memory across invocations, no per-session storage of substrate-relevant state treated as authoritative, no in-weight memory of substrate content treated as the answer to “what is the case”).
4. The LLM does not exercise authority over substrate content (no silent overwrite, no collapse of preserved contradictions, no modification of orchestration rules; it executes within them).
5. LLM operations that affect substrate state are recorded in the substrate as substrate content, with attribution identifying them as LLM-authored and associating them with the orchestration rule under which they were produced.

A system that fails any of (1)–(5) for any LLM operation may instantiate some other architectural role for its LLMs, but does not implement the AI-as-substrate-mediator commitment in the CKS sense.

8. Why naming this role matters

Two failure modes become explicitly identifiable once the mediator role is named, both ruled out implicitly by the source paper without a directly citable prohibition.

The first is *LLM-as-authority*. Implementations that grant the LLM the right to overwrite, collapse, or modify rules without human action may be useful for other purposes, and may even be governed in some other sense, but they are not CKS-coherent: they violate property (4) and undo the work the human-governed and conflict-preservation commitments do. The second is *LLM-as-state-holder*. Implementations that hold substrate-relevant state outside the substrate — in agent memory, in per-session storage treated as authoritative, in in-weight memory the system relies on as a source of truth — produce the failure mode the source paper names at §6.2 as *context rot*: capacity overflow, compaction loss, and goal drift under repeated compression of state the substrate could have held instead. The architectural property at issue is *where the state lives*, not *how it is managed*; the failure mode applies regardless of how technically sophisticated the holding mechanism is.

Naming the mediator role does not change what CKS is. It makes what CKS is independently citable, and it makes the boundaries of what an LLM is authorized to do in a CKS system architectural rather than implicit. Subsequent work that adopts, extends, composes, or argues against the CKS pattern should use AI-as-substrate-mediator in the sense formalized here. Subsequent work that grants LLMs authority over substrate content, or that holds substrate-relevant state outside the substrate, is using a different architectural role, and the difference should be named.

———

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Mediator, Not Authority: The Architectural Role of LLMs in the Coordination Knowledge Substrate Pattern.* 24 April 2026. ORCID: 0009-0004-8065-3235.